CSD_05 — MCQ Bank & Descriptive Question Bank (Full CS Degree Syllabus)

> Covers the **MCQ paper** (100 × 2 = 200 marks, −1/3 negative marking) and the **descriptive Paper-I / Paper-II** (each 100 marks, Sec A 8-of-10 × 5, Sec B 3-of-5 × 10, Sec C 2-of-4 × 15), across the whole CS (Degree) syllabus in `OFFICIAL_SYLLABUS.md` §3 — Basics & Programming, Data Structures & Algorithms, Logic Design, Computer Organization & Architecture, System Software & Compilers, Mobile Applications (Paper-I); Operating Systems, Software Engineering, DBMS, Computer Networks, Web Technologies, AI, Machine Learning (Paper-II).
> This file is a **drilling tool**, not a teaching one — it assumes you have read the topic files (`CSD_00`–`CSD_04` and the relevant `CORE_*` files) and are now testing recall and exam-writing speed. Difficulty is calibrated to the solved 2025 CS-Degree papers in `_raw/existing_txt/`.

How to use this file

- [ ] Attempt each MCQ block of 10 **closed-book**, covering the green answer box under each question; reveal it only after you have committed to an answer.
- [ ] For every five-mark skeleton, write the actual bullet points from memory in under 4 minutes; for ten-mark in under 8; for fifteen-mark in under 12 — that is your real per-question time budget (180 minutes ÷ 13 questions answered ≈ 14 min each, with slack for reading/choosing).
- [ ] Re-attempt only the MCQs you got wrong or guessed, weekly, until the block is clean.

The negative-marking rule — read this before you touch the MCQ paper

> **Check on the day:** the 2025 booklet printed **"−⅓ mark"** per wrong answer. If that is the rule, a blind guess is worth **+0.25** on average rather than 0, so attempting everything is slightly better still. The table below uses the stricter reading (−0.667).

MCQ is 100 questions × 2 marks = 200 marks; each **wrong** answer costs **−1/3 mark actual**, i.e. −1/3 × 2 = **−0.667 marks** (some official notations state the penalty as −1/3 of the question's own mark value; either way, treat it as **losing a third of what a correct answer would have earned**). An **unattempted** question costs exactly 0.

The maths that decides whether to guess:

Strategy	Expected value per question (4 options, uniform prior)
Leave blank	0
Guess blind among all 4 options	(1/4)(+2) + (3/4)(−2/3) = 0.5 − 0.5 = **0** (exact break-even)
Guess among 3 options (1 eliminated)	(1/3)(+2) + (2/3)(−2/3) = 0.667 − 0.444 = **+0.222** (positive)
Guess among 2 options (2 eliminated)	(1/2)(+2) + (1/2)(−2/3) = 1 − 0.333 = **+0.667** (strongly positive)
Answer with genuine confidence	Expected value → +2 as confidence → certainty

The rule in one line: a pure blind 4-way guess is exactly break-even — it costs you nothing in
expectation, but it does add variance for no expected gain, so there is no reason to

blind-guess.
The moment you can eliminate even one wrong option, the guess turns strictly profitable — always
guess once you've eliminated at least one distractor. Never leave a question blank just because
you are unsure between three options; leave it blank only when you truly have zero information to eliminate any option (rare — most "trap" options in this paper are eliminable by basic reasoning: wrong units, wrong data type, wrong direction of a comparison).

PART 1 — MCQ BANK (120 questions, answer below each question)

Block 1 — Programming in C/C++ (Q1–10)

1. What is the output of `printf("%d", 7 / 2);` in C?
A) 3.5 B) 3 C) 4 D) Compile error

> **Ans: B) 3** — Integer division in C truncates toward zero: 7/2 = 3
> *Trap:* A assumes float division

2. Which header is required for `malloc()` in C?
A) `<stdio.h>` B) `<stdlib.h>` C) `<memory.h>` D) `<conio.h>`

> **Ans: B) `<stdlib.h>`** — `malloc`, `calloc`, `free` declared in `stdlib.h`
> *Trap:* `stdio.h` is I/O only

3. What does `strcmp("abc", "abc")` return?
A) 1 B) −1 C) 0 D) The address of the string

> **Ans: C) 0** — Identical strings → `strcmp` returns 0
> *Trap:* 1/−1 are for "greater/less than" cases

4. What is the scope of a `static` local variable in C?
A) Global to the whole program B) Local to the function, but retains value between calls C) Local to the block only D) File scope only

> **Ans: B) Local to the function, but retains value between calls** — `static` locals persist across calls but stay function-scoped
> *Trap:* Confused with global scope

5. In C++, which mechanism allows a derived class to provide its own version of a base class function?
A) Overloading B) Overriding (via virtual functions) C) Templates D) Friend functions

> **Ans: B) Overriding (via virtual functions)** — Runtime polymorphism via virtual functions is overriding
> *Trap:* Overloading is compile-time, same name/diff signature

6. What is the size (in bytes, typical 32/64-bit system) of a `char` in C?
A) 1 B) 2 C) 4 D) 8

> **Ans: A) 1** — `char` is 1 byte (guaranteed by the C standard)

7. Which of these is NOT a valid C storage class?
A) `auto` B) `register` C) `extern` D) `dynamic`

> **Ans: D) `dynamic`** — `auto`, `register`, `static`, `extern` are the four; "dynamic" is not a C keyword

8. What does the `volatile` keyword signify in C?
A) The variable is thread-safe B) The compiler must not optimize away reads/writes to this variable C) The variable is stored in ROM D) The variable cannot change

> **Ans: B) The compiler must not optimize away reads/writes to this variable** — `volatile` prevents compiler optimisation of reads/writes (value may change outside program flow)
> *Trap:* Thread-safety is not what `volatile` guarantees in C

9. In C++, what is function overloading?
A) Same function name with different implementations at runtime B) Same function name with different parameter lists resolved at compile time C) A function that calls itself D) A virtual function

> **Ans: B) Same function name with different parameter lists resolved at compile time** — Overloading resolved at compile time by signature
> *Trap:* A describes overriding (runtime)

10. What is a dangling pointer?
A) A pointer that has never been initialised B) A pointer that points to memory that has been freed/deallocated C) A NULL pointer D) A pointer to a function

> **Ans: B) A pointer that points to memory that has been freed/deallocated** — Freed memory still pointed to = dangling pointer
> *Trap:* Uninitialised pointer is a "wild" pointer, a related but distinct term

Block 2 — Data Structures (Q11–20)

11. Which data structure uses LIFO (Last In First Out) order?
A) Queue B) Stack C) Linked list D) Tree

> **Ans: B) Stack** — Stack = LIFO by definition
> *Trap:* Queue is FIFO

12. The worst-case time complexity of searching in a balanced binary search tree is:
A) O(1) B) O(log n) C) O(n) D) O(n log n)

> **Ans: B) O(log n)** — Balanced BST height is O(log n), so search is O(log n)
> *Trap:* Unbalanced BST could be O(n), but "balanced" is specified

13. Which sorting algorithm is stable AND has Θ(n log n) worst-case complexity?
A) Quick sort B) Heap sort C) Merge sort D) Selection sort

> **Ans: C) Merge sort** — Merge sort: stable and Θ(n log n) worst case
> *Trap:* Quick sort worst is Θ(n^2); heap sort is Θ(n log n) but NOT stable

14. In a hash table, when two keys map to the same slot, this is called:
A) Overflow B) Collision C) Rehashing D) Aliasing

> **Ans: B) Collision** — Two keys → same slot = collision
> *Trap:* Overflow is a related but distinct term (bucket full)

15. A queue implemented using two stacks needs how many stacks reversed to dequeue?
A) Not possible B) One C) Two D) It depends on data

> **Ans: C) Two** — Classic "queue using two stacks" needs both stacks
> *Trap:* One-stack solutions do not give O(1) amortised dequeue+enqueue split cleanly
> ⚠️ Loosely worded. Strictly, *one* stack is reversed (its items are popped into the other), but the intended answer is (C), because the technique needs both stacks.

16. What is the time complexity of inserting at the head of a singly linked list?
A) O(1) B) O(log n) C) O(n) D) $O(n^2)$

> **Ans: A) O(1)** — Insert at head is a fixed number of pointer updates: O(1)
> *Trap:* Insert at tail (no tail pointer) would be O(n)

17. Which traversal of a binary search tree yields elements in sorted order?
A) Preorder B) Inorder C) Postorder D) Level order

> **Ans: B) Inorder** — Inorder (Left-Root-Right) on a BST yields sorted order
> *Trap:* Preorder/postorder do not sort

18. A complete binary tree with n nodes has height:
A) O(n) B) O(log n) C) $O(n^2)$ D) $O(\sqrt{n})$

> **Ans: B) O(log n)** — Complete binary tree height is O(log n) by definition

19. What is the auxiliary space complexity of standard (non-in-place) merge sort?
A) O(1) B) O(log n) C) O(n) D) $O(n^2)$

> **Ans: C) O(n)** — Standard merge sort needs an O(n) auxiliary buffer for merging
> *Trap:* In-place variants exist but are non-standard/slower

20. In a min-heap, where is the smallest element always located?
A) Last leaf B) Root C) Rightmost node D) Undetermined

> **Ans: B) Root** — Min-heap property: root is always the minimum
> *Trap:* Heaps are not sorted arrays; only the root guarantee holds

Block 3 — Algorithm Analysis (Q21–30)

21. What is the time complexity of linear search in the worst case?
A) O(1) B) O(log n) C) O(n) D) $O(n^2)$

> **Ans: C) O(n)** — Linear search worst case scans all n elements: O(n)

22. Which notation gives an asymptotically tight bound?
A) Big-O B) Big-Omega C) Big-Theta D) little-o

> **Ans: C) Big-Theta** — Θ gives matching upper AND lower bound = tight
> *Trap:* O is upper only, Ω is lower only

23. The recurrence $T(n) = 2T(n/2) + n$ solves to:
A) $\Theta(n)$ B) $\Theta(n \log n)$ C) $\Theta(n^2)$ D) $\Theta(\log n)$

> **Ans: B) Θ(n log n)** — Master theorem case 2: a=2,b=2,f(n)=n, e=1 → Θ(n log n)

24. Quick sort's worst-case time complexity occurs when the input array is:
A) Random B) Already sorted (with first/last-element pivot) C) All equal elements only D) Reverse of random

> **Ans: B) Already sorted (with first/last-element pivot)** — Already-sorted input with a bad (first/last) pivot choice gives 0/(n-1) splits every time
> *Trap:* Random input gives the average case, not worst

25. Which paradigm builds a solution by always taking the locally optimal choice?
A) Divide and conquer B) Dynamic programming C) Greedy D) Backtracking

> **Ans: C) Greedy** — Greedy = locally optimal choice each step, no backtracking
> *Trap:* DP evaluates all options via subproblems

26. Dynamic programming is applicable when a problem exhibits:
A) Greedy-choice property only B) Optimal substructure and overlapping subproblems C) NP-completeness D) No recursive structure

> **Ans: B) Optimal substructure and overlapping subproblems** — DP requires optimal substructure AND overlapping subproblems
> *Trap:* Greedy-choice property belongs to greedy, not DP

27. Which of these problems is solved optimally by a greedy algorithm?
A) 0/1 Knapsack B) Fractional Knapsack C) Longest Common Subsequence D) Travelling Salesman

> **Ans: B) Fractional Knapsack** — Fractional knapsack: greedy on value/weight ratio is provably optimal
> *Trap:* 0/1 knapsack fails greedy, needs DP

28. A problem is in NP if:
A) It can be solved in polynomial time B) A proposed solution can be verified in polynomial time C) It cannot be solved at all D) It requires exponential space

> **Ans: B) A proposed solution can be verified in polynomial time** — NP = solutions verifiable in polynomial time
> *Trap:* Solvable in poly time defines P, not NP

29. Which traversal algorithm uses a queue internally?
A) DFS B) BFS C) Inorder traversal D) Postorder traversal

> **Ans: B) BFS** — BFS explicitly uses a queue (level-by-level)
> *Trap:* DFS uses a stack/recursion

30. Strassen's matrix multiplication algorithm reduces the number of multiplications per recursive step from 8 to:
A) 6 B) 7 C) 5 D) 4

> **Ans: B) 7** — Strassen reduces 8 multiplications to 7 (at the cost of more additions)

Block 4 — Digital Logic & Number Systems (Q31–40)

31. The 2's complement of the 8-bit binary number 00001010 is:
A) 11110101 B) 11110110 C) 11110100 D) 00001010

> **Ans: B) 11110110** — 00001010 (=10) → invert → 11110101 → add 1 → **11110110**. Option (a) is only the 1's complement.

32. Which gate produces an output of 1 only when both inputs differ?
A) AND B) OR C) XOR D) NAND

> **Ans: C) XOR** — XOR outputs 1 when inputs differ
> *Trap:* AND/OR do not have this "difference detector" property

33. How many flip-flops are needed to build a MOD-16 counter?
A) 2 B) 4 C) 8 D) 16

> **Ans: B) 4** — MOD-16 = 2^4, needs 4 flip-flops
> *Trap:* 2 flip-flops only gives MOD-4

34. NAND gate is called a "universal gate" because:
A) It is the fastest gate B) Any Boolean function can be implemented using only NAND gates C) It has the lowest power consumption D) It was invented first

> **Ans: B) Any Boolean function can be implemented using only NAND gates** — NAND (and NOR) alone can implement any Boolean function — universal gate
> *Trap:* Not about speed or history

35. The hexadecimal equivalent of decimal 255 is:
A) FF B) FE C) 100 D) F0

> **Ans: A) FF** — 255 decimal = FF hex (15×16+15)

36. A K-map is used primarily for:
A) Sequential circuit design B) Minimising Boolean expressions C) Memory addressing D) Clock synchronisation

> **Ans: B) Minimising Boolean expressions** — Karnaugh maps minimise Boolean/SOP-POS expressions
> *Trap:* Not used for sequential design directly

37. A JK flip-flop with J=1, K=1 on a clock edge will:
A) Set to 1 B) Reset to 0 C) Toggle its previous state D) Remain unchanged (no-change/hold)

> **Ans: C) Toggle its previous state** — JK flip-flop with J=K=1 toggles on clock edge
> *Trap:* J=K=0 is hold; J=1,K=0 is set; J=0,K=1 is reset

38. A multiplexer with n select lines can choose among how many inputs?
A) n B) n^2 C) 2n D) 2^n

> **Ans: D) 2^n** — n select lines choose among 2^n inputs
> *Trap:* Common MCQ trap swapping n and 2^n

39. In 1's complement representation, what causes the "end-around carry" adjustment?
A) Overflow of the sign bit only B) A carry-out from the MSB, added back to the LSB C) Underflow D)

Nothing — 1's complement has no such adjustment

> **Ans: B) A carry-out from the MSB, added back to the LSB** — 1's complement addition: carry-out from MSB wraps around and adds to LSB
> *Trap:* 2's complement addition simply discards the final carry

40. A half adder produces which two outputs?
A) Sum and Carry B) Sum and Borrow C) Carry and Overflow D) Product and Remainder

> **Ans: A) Sum and Carry** — Half adder: Sum and Carry outputs only (no carry-in)
> *Trap:* Full adder additionally takes a carry-in

Block 5 — Computer Organization & Architecture (Q41–50)

41. Which register holds the address of the next instruction to be fetched?
A) Accumulator B) Program Counter (PC) C) Instruction Register (IR) D) Memory Address Register (MAR)

> **Ans: B) Program Counter (PC)** — Program Counter holds address of the NEXT instruction
> *Trap:* IR holds the CURRENT instruction being decoded

42. Cache memory is used primarily to:
A) Increase disk storage B) Bridge the speed gap between CPU and main memory C) Store the operating system permanently D) Manage virtual memory pages

> **Ans: B) Bridge the speed gap between CPU and main memory** — Cache bridges CPU-memory speed gap
> *Trap:* Not disk-related

43. DMA (Direct Memory Access) allows:
A) The CPU to access memory faster B) Peripheral devices to transfer data to/from memory without CPU intervention for each word C) Two CPUs to share memory D) Memory to access the disk directly

> **Ans: B) Peripheral devices to transfer data to/from memory without CPU intervention for each word** — DMA transfers data directly without per-word CPU involvement
> *Trap:* CPU is freed up, not sped up directly

44. Which addressing mode uses the operand's address stored in a register?
A) Immediate B) Direct C) Register indirect D) Indexed

> **Ans: C) Register indirect** — Register indirect: register holds the address, not the value
> *Trap:* Direct addressing stores the address in the instruction itself

45. The technique of overlapping fetch, decode and execute of successive instructions is called:
A) Multiprogramming B) Pipelining C) Multitasking D) Interleaving

> **Ans: B) Pipelining** — Overlapping instruction stages = pipelining
> *Trap:* Multiprogramming/multitasking are OS-level concepts, different layer

46. Which memory is the fastest but most expensive per bit?
A) Main memory (RAM) B) Cache memory (built from registers/SRAM) C) Magnetic disk D) Optical disk

> **Ans: B) Cache memory (built from registers/SRAM)** — Cache (SRAM) is faster and costlier per bit than RAM
> *Trap:* RAM is cheaper/slower than cache

47. Which of these is a cause of a structural hazard in a pipeline?
A) Data dependency between instructions B) Two instructions needing the same hardware resource simultaneously C) A branch instruction changing control flow D) An interrupt

> **Ans: B) Two instructions needing the same hardware resource simultaneously** — Structural hazard = resource contention (e.g. one memory port needed by two stages)
> *Trap:* Data hazard is about data dependency

48. What does "memory interleaving" primarily improve?
A) Memory capacity B) Memory access bandwidth by allowing overlapped access across banks C) Memory security D) Cache coherence

> **Ans: B) Memory access bandwidth by allowing overlapped access across banks** — Interleaving spreads addresses across banks for overlapped, higher-bandwidth access
> *Trap:* Not a capacity or security feature

49. In the immediate addressing mode, the operand is:
A) In a register B) In memory, address given directly C) Part of the instruction itself D) In memory, address computed from a base register

> **Ans: C) Part of the instruction itself** — Immediate addressing: the operand's actual value is embedded in the instruction
> *Trap:* Direct addressing gives an address, not the value itself

50. Which unit of the CPU performs arithmetic and logical operations?
A) Control Unit B) ALU C) Cache D) MMU

> **Ans: B) ALU** — ALU = Arithmetic Logic Unit performs computation
> *Trap:* Control Unit directs, does not compute

Block 6 — System Software & Compilers (Q51–60)

51. Which phase of a compiler builds the symbol table entries and checks type compatibility?
A) Lexical analysis B) Syntax analysis C) Semantic analysis D) Code generation

> **Ans: C) Semantic analysis** — Semantic analysis does type checking and symbol table use/verification
> *Trap:* Lexical analysis only tokenises
> ⚠️ Symbol-table entries are first created during lexical/syntax analysis; the semantic phase uses and verifies them and does the type checking. The intended answer is (C).

52. A "token" in compiler terminology is produced by:
A) The parser B) The lexical analyser (scanner) C) The code generator D) The linker

> **Ans: B) The lexical analyser (scanner)** — The lexical analyser (scanner) groups characters into tokens
> *Trap:* Parser consumes tokens, does not produce them

53. Which parsing technique processes input from left to right, building the rightmost derivation in reverse?
A) LL parsing B) LR parsing C) Recursive descent D) Operator precedence

> **Ans: B) LR parsing** — LR parsing builds a rightmost derivation in reverse, scanning left to right
> *Trap:* LL builds leftmost derivation

54. A macro, as used by an assembler, is:
A) A hardware instruction B) A named template of code expanded inline at each call, at assembly/compile time C) A runtime function call D) A type of loader

> **Ans: B) A named template of code expanded inline at each call, at assembly/compile time** — A macro is expanded inline at assembly/compile time by the preprocessor/assembler
> *Trap:* Not a runtime call

55. The main job of a linker is to:
A) Convert source code to assembly B) Combine object modules and resolve external references into an executable C) Load a program into memory for execution D) Detect syntax errors

> **Ans: B) Combine object modules and resolve external references into an executable** — Linker combines object files, resolves external symbol references
> *Trap:* Loader places the executable in memory; different stage

56. Which loader scheme relocates a program's addresses at load time, allowing it to be loaded anywhere in memory?
A) Absolute loader B) Relocating loader C) Direct linking loader D) Bootstrap loader

> **Ans: B) Relocating loader** — Relocating loader adjusts addresses so the program can load anywhere
> *Trap:* Absolute loader requires a fixed load address

57. Dead code elimination and constant folding are examples of:
A) Lexical analysis techniques B) Code optimisation techniques C) Syntax error recovery D) Symbol table operations

> **Ans: B) Code optimisation techniques** — Both are classic optimisation-phase techniques
> *Trap:* Not lexical or symbol-table operations

58. Which grammar class corresponds to context-free languages, relevant to most programming language syntax?
A) Type 0 (unrestricted) B) Type 1 (context-sensitive) C) Type 2 (context-free) D) Type 3 (regular)

> **Ans: C) Type 2 (context-free)** — Most PL syntax is context-free (Type 2 in the Chomsky hierarchy)
> *Trap:* Regular (Type 3) cannot express nested structures like balanced parentheses

59. An "editor" in the context of system software refers to:
A) A program that translates source code B) A tool for creating/modifying source text files C) A device driver D) A scheduling algorithm

> **Ans: B) A tool for creating/modifying source text files** — An editor creates/modifies source text
> *Trap:* Not a translator or scheduler

60. A three-address code intermediate representation typically has the form:
A) `x = y op z` B) `x = y` C) A parse tree only D) Raw machine code

> **Ans: A)** **`x = y op z`** — Three-address code form: one operator, at most two operands, one result
> *Trap:* `x=y` is a simpler special case, not the general form

Block 7 — Operating Systems (Q61–70)

61. Which CPU scheduling algorithm can cause starvation of long processes?
A) FCFS B) Round Robin C) Shortest Job First (SJF) D) None of these

> **Ans: C) Shortest Job First (SJF)** — SJF can starve long jobs if short jobs keep arriving
> *Trap:* FCFS/RR do not starve by job length

62. A deadlock can occur only if all of the following conditions hold simultaneously EXCEPT:
A) Mutual exclusion B) Hold and wait C) Preemption D) Circular wait

> **Ans: C) Preemption** — The four necessary conditions are Mutual Exclusion, Hold-and-Wait, **No** Preemption, Circular Wait — so "Preemption" (allowing it) breaks the deadlock condition set; it is the one that must NOT hold for deadlock
> *Trap:* This question tests whether you know "No Preemption" (not "Preemption") is the actual condition

63. Which memory management technique suffers from external fragmentation?
A) Paging B) Segmentation with variable-sized partitions C) Fixed-size paging only D) None

> **Ans: B) Segmentation with variable-sized partitions** — Variable-sized segmentation/partitions can leave unusable external gaps
> *Trap:* Fixed-size paging causes internal, not external, fragmentation

64. The "belady's anomaly" refers to:
A) A CPU scheduling paradox B) Increasing page faults despite increasing the number of frames, under FIFO page replacement C) A deadlock detection failure D) A file system corruption bug

> **Ans: B) Increasing page faults despite increasing the number of frames, under FIFO page replacement** — Belady's anomaly: FIFO can have MORE page faults with MORE frames — counter-intuitive
> *Trap:* Not related to scheduling or deadlock

65. Which page replacement algorithm is provably optimal (minimises page faults) but not implementable in practice?
A) FIFO B) LRU C) Optimal (OPT/Belady's algorithm) D) Second-chance

> **Ans: C) Optimal (OPT/Belady's algorithm)** — Optimal (Belady's) algorithm needs future knowledge — theoretical benchmark only
> *Trap:* LRU/FIFO are practical approximations

66. A critical section problem solution must satisfy:
A) Mutual exclusion, Progress, Bounded waiting B) Only mutual exclusion C) Only fairness D) Deadlock freedom alone, without mutual exclusion

> **Ans: A) Mutual exclusion, Progress, Bounded waiting** — Mutual exclusion + Progress + Bounded waiting are the three required properties
> *Trap:* Fairness alone is insufficient; deadlock-freedom alone is insufficient

67. Which is a user-level thread's main disadvantage compared to kernel-level threads?
A) Faster context switch B) A blocking system call blocks the entire process C) Requires no kernel support D) None — user threads have no disadvantages

> **Ans: B) A blocking system call blocks the entire process** — User-level threads: kernel sees only the process, so one blocking syscall blocks all threads in it
> *Trap:* Kernel threads don't have this issue, at the cost of heavier context switch

68. What is thrashing?
A) A CPU scheduling technique B) Excessive paging activity such that the system spends more time swapping than executing C) A disk-scheduling algorithm D) A file allocation method

> **Ans: B) Excessive paging activity such that the system spends more time swapping than executing** — Thrashing = system busier paging than computing
> *Trap:* Not a scheduling or disk algorithm term

69. RAID level 1 primarily provides:
A) Striping for performance only, no redundancy B) Mirroring for redundancy C) Parity-based redundancy across all disks D) No fault tolerance

> **Ans: B) Mirroring for redundancy** — RAID 1 = mirroring (full redundant copy)
> *Trap:* RAID 0 is striping with no redundancy

70. Which file allocation method suffers most from external fragmentation on disk?
A) Linked allocation B) Indexed allocation C) Contiguous allocation D) FAT-based allocation only

> **Ans: C) Contiguous allocation** — Contiguous allocation leaves gaps between used regions as files grow/shrink = external fragmentation
> *Trap:* Linked/indexed allocation avoid this by not requiring contiguous blocks

Block 8 — Software Engineering (Q71–78)

71. Which SDLC model is purely sequential, with no going back to a previous phase?
A) Spiral model B) Waterfall model (classic form) C) Agile model D) Prototype model

> **Ans: B) Waterfall model (classic form)** — Classic waterfall: strictly sequential, no iteration back
> *Trap:* Spiral/Agile are iterative by design

72. The Spiral model is best characterised by:
A) Strict sequential phases B) Repeated cycles combining prototyping with risk analysis C) No planning phase D) Testing done only once at the end

> **Ans: B) Repeated cycles combining prototyping with risk analysis** — Spiral = repeated risk-driven prototyping cycles
> *Trap:* Waterfall has no repeated cycles

73. Cyclomatic complexity is a metric used to measure:
A) Lines of code only B) The structural/logical complexity of a program via its control flow graph C) Memory usage D) Team size needed

> **Ans: B) The structural/logical complexity of a program via its control flow graph** — Cyclomatic complexity (McCabe) measures independent paths through a control flow graph
> *Trap:* Not about code length or team

74. Which testing technique tests a module without knowledge of its internal code structure?
A) White-box testing B) Black-box testing C) Unit testing only D) Structural testing

> **Ans: B) Black-box testing** — Black-box testing = behaviour only, no internal code knowledge
> *Trap:* White-box explicitly uses code structure

75. What is the primary goal of "verification" in software QA, as distinct from "validation"?
A) Are we building the product right? (per the requirements) B) Are we building the right product? (does it meet user needs) C) Is the product fast enough? D) Is the product secure?

> **Ans: A) Are we building the product right? (per the requirements)** — Verification = "building it right" (per spec); Validation = "building the right thing" (per user need)
> *Trap:* B describes validation, the paired but different term

76. Which is NOT one of the standard software testing levels?
A) Unit testing B) Integration testing C) System testing D) Compilation testing

> **Ans: D) Compilation testing** — Unit, Integration, System (and Acceptance) are standard levels; "Compilation testing" is not a recognised testing level

77. COCOMO is a model used for:
A) Database normalisation B) Software cost/effort estimation C) Network routing D) Memory allocation

> **Ans: B) Software cost/effort estimation** — COCOMO = COnstructive COst MOdel, for effort/cost estimation
> *Trap:* Not a DB or network model

78. In requirements analysis, a "functional requirement" describes:
A) How fast the system must respond B) What the system must do (specific behaviour/function) C) The programming language to use D) The hardware budget

> **Ans: B) What the system must do (specific behaviour/function)** — Functional requirement = what the system does
> *Trap:* A describes a non-functional (performance) requirement

Block 9 — DBMS (Q79–86)

79. Which SQL clause is used to filter groups after a `GROUP BY`?
A) WHERE B) HAVING C) ORDER BY D) FILTER

> **Ans: B) HAVING** — HAVING filters aggregated groups; WHERE filters rows before grouping
> *Trap:* WHERE cannot reference aggregate functions directly

80. A relation is in 2NF if it is in 1NF and:
A) Has no transitive dependency B) Has no partial dependency of any non-key attribute on the primary key C) Has no multivalued dependency D) Has a single candidate key only

> **Ans: B) Has no partial dependency of any non-key attribute on the primary key** — 2NF: no partial dependency on part of a composite key
> *Trap:* Transitive dependency is the 3NF condition

81. Which SQL keyword removes duplicate rows from a result set?
A) UNIQUE B) DISTINCT C) FILTER D) GROUP

> **Ans: B) DISTINCT** — `SELECT DISTINCT` removes duplicate rows
> *Trap:* `UNIQUE` is a constraint, not a query keyword for this purpose

82. A foreign key constraint enforces:
A) Uniqueness within its own table B) Referential integrity — that the value must exist in the referenced table (or be null) C) That the column cannot be null D) That the column is auto-incremented

> **Ans: B) Referential integrity — that the value must exist in the referenced table (or be null)** — FK enforces that referenced values exist (or are null) — referential integrity
> *Trap:* Not about uniqueness within its own table

83. Which SQL statement modifies existing rows in a table?
A) INSERT B) UPDATE C) ALTER D) CREATE

> **Ans: B) UPDATE** — UPDATE modifies existing row values
> *Trap:* INSERT adds new rows; ALTER changes schema

84. A "candidate key" is:
A) Any key chosen arbitrarily by the DBA B) A minimal set of attributes that can uniquely identify a tuple C) A foreign key from another table D) Always the primary key

> **Ans: B) A minimal set of attributes that can uniquely identify a tuple** — Candidate key = minimal unique-identifying attribute set
> *Trap:* Not arbitrary, and not necessarily the chosen primary key

85. Which type of SQL join returns all rows from the left table and matched rows from the right, with NULLs where there is no match?
A) INNER JOIN B) LEFT OUTER JOIN C) RIGHT OUTER JOIN D) CROSS JOIN

> **Ans: B) LEFT OUTER JOIN** — LEFT OUTER JOIN keeps all left rows, NULLs for unmatched right side
> *Trap:* RIGHT OUTER JOIN is the mirror case

86. A transaction's "isolation" property in ACID ensures:
A) All operations complete or none do B) The database remains in a consistent state C) Concurrent transactions do not interfere with each other's intermediate states D) Committed changes survive a crash

> **Ans: C) Concurrent transactions do not interfere with each other's intermediate states** — Isolation = concurrent transactions don't see each other's partial/intermediate state
> *Trap:* Atomicity is "all or nothing"; Durability is "survives crash"

Block 10 — Computer Networks (Q87–94)

87. Which OSI layer is responsible for routing packets between different networks?
A) Data Link layer B) Network layer C) Transport layer D) Session layer

> **Ans: B) Network layer** — Network layer (e.g. IP) handles routing across networks
> *Trap:* Data Link handles same-network framing

88. TCP differs from UDP primarily in that TCP is:
A) Faster but unreliable B) Connection-oriented and reliable (with acknowledgment/retransmission)

C) Used only for streaming D) Stateless

> **Ans: B) Connection-oriented and reliable (with acknowledgment/retransmission)** — TCP: connection-oriented, reliable, with ACK/retransmit
> *Trap:* UDP is connectionless, faster, unreliable

89. A subnet mask of 255.255.255.0 corresponds to a CIDR prefix of:
A) /16 B) /24 C) /8 D) /32

> **Ans: B) /24** — /24 = 24 network bits = 255.255.255.0
> *Trap:* /16 would be 255.255.0.0

90. Which device operates at the Data Link layer, forwarding frames based on MAC addresses?
A) Hub B) Switch C) Router D) Repeater

> **Ans: B) Switch** — Switch forwards frames by MAC address (Layer 2)
> *Trap:* Hub is a dumb broadcaster; Router works at Layer 3 by IP

91. DNS primarily performs:
A) Encryption of network traffic B) Translation of domain names to IP addresses C) Routing table calculation D) Error correction in transmission

> **Ans: B) Translation of domain names to IP addresses** — DNS = Domain Name System, resolves names to IPs
> *Trap:* Not encryption or routing computation

92. Which of these is a Class C private IP address range?
A) 10.0.0.0 – 10.255.255.255 B) 172.16.0.0 – 172.31.255.255 C) 192.168.0.0 – 192.168.255.255 D) 127.0.0.0 – 127.255.255.255

> **Ans: C) 192.168.0.0 – 192.168.255.255** — 192.168.0.0/16 is the Class C-range private block
> *Trap:* 10.x is Class A private; 172.16–172.31 is Class B private

93. The three-way handshake used to establish a TCP connection is:
A) SYN, SYN, ACK B) SYN, SYN-ACK, ACK C) ACK, SYN, FIN D) SYN, ACK, FIN

> **Ans: B) SYN, SYN-ACK, ACK** — TCP handshake: SYN → SYN-ACK → ACK
> *Trap:* Order and combined SYN-ACK packet matter

94. Which topology has every node connected directly to every other node?
A) Star B) Bus C) Mesh (full mesh) D) Ring

> **Ans: C) Mesh (full mesh)** — Full mesh: every node directly connects to every other
> *Trap:* Star routes through a central hub; ring is a closed loop

Block 11 — Web Technologies (HTML/CSS/PHP) (Q95–104)

95. In an HTML form, which attribute determines where the form data is sent?
A) `method` B) `action` C) `target` D) `name`

> **Ans: B) `action`** — `action` attribute names the form's target URL
> *Trap:* `method` says GET/POST, not destination

96. Which PHP superglobal array holds data submitted via the POST method?
A) `$_GET` B) `$_POST` C) `$_REQUEST` only D) `$_SESSION`

> **Ans: B)** **`$_POST`** — `$_POST` holds POST-submitted form data
> *Trap:* `$_GET` is for query-string data

97. Which HTML tag is used to define an unordered (bulleted) list?
A) `<ol>` B) `<ul>` C) `<dl>` D) `<list>`

> **Ans: B)** **`<ul>`** — `<ul>` = unordered list (bulleted)
> *Trap:* `<ol>` is ordered (numbered)

98. In PHP, which comparison operator checks both value and type equality?
A) `==` B) `===` C) `=` D) `<=>`

> **Ans: B)** **`===`** — `===` checks value AND type (strict)
> *Trap:* `==` is loose/type-juggling

99. Where does data sent via the GET method appear?
A) In the HTTP request body only B) Appended to the URL as a query string C) In a cookie automatically D) Nowhere visible

> **Ans: B) Appended to the URL as a query string** — GET data is appended to the URL as `?key=value` pairs
> *Trap:* Not sent silently in a cookie

100. In PHP, session data is stored:
A) In the client's browser as plain text B) On the server, with only a session ID held client-side C) In the HTML source code D) In the URL always

> **Ans: B) On the server, with only a session ID held client-side** — Session data lives server-side; only the session ID token sits in a client cookie
> *Trap:* Not stored as plaintext client-side

101. What is the correct file extension for a server to invoke the PHP interpreter?
A) `.html` B) `.php` C) `.htm` D) `.js`

> **Ans: B)** **`.php`** — `.php` extension triggers server-side PHP processing
> *Trap:* `.html`/`.htm` are served as static text, ignoring any PHP markup inside

102. Which CSS property controls the space between an element's border and its content?
A) `margin` B) `padding` C) `border-spacing` D) `outline`

> **Ans: B)** **`padding`** — `padding` = space between content and border
> *Trap:* `margin` = space OUTSIDE the border, between elements

103. The `alt` attribute of an `<img>` tag is used for:
A) Setting image alignment B) Alternate text shown if the image fails to load / read by screen readers C) Setting image width D) Compressing the image

> **Ans: B) Alternate text shown if the image fails to load / read by screen readers** — `alt` = fallback text / accessibility description
> *Trap:* Not for sizing or compression

104. Which HTTP status code indicates the requested resource was not found?
A) 200 B) 301 C) 404 D) 500

> **Ans: C) 404** — 404 = Not Found
> *Trap:* 500 = server error; 200 = success; 301 = redirect

Block 12 — Mobile Applications (Q105–112)

105. Which of the four Android components has no user interface and runs in the background?
A) Activity B) Service C) Content Provider D) Fragment (not one of the four)

> **Ans: B) Service** — Service has no UI, runs in background
> *Trap:* Activity has UI; Fragment is not one of the four core components

106. The Android callback invoked when an activity becomes fully invisible is:
A) `onPause()` B) `onStop()` C) `onDestroy()` D) `onCreate()`

> **Ans: B) `onStop()`** — `onStop()` fires when the activity is no longer visible at all
> *Trap:* `onPause()` fires while partially visible/covered

107. The minimum recommended touch target size on Android is approximately:
A) 24×24 dp B) 48×48 dp C) 100×100 dp D) 16×16 dp

> **Ans: B) 48×48 dp** — 48×48 dp is Android's minimum recommended touch target
> *Trap:* iOS uses 44×44 pt, a different unit and OS

108. Which file in an Android project declares all its components and permissions?
A) `build.gradle` B) `AndroidManifest.xml` C) `strings.xml` D) `MainActivity.java`

> **Ans: B) `AndroidManifest.xml`** — `AndroidManifest.xml` declares components, permissions, SDK versions
> *Trap:* `build.gradle` is the build config file, a different purpose

109. Flutter compiles application code to:
A) Interpreted JavaScript B) Native ARM machine code via its own rendering engine C) A hybrid WebView wrapper only D) Bytecode requiring a JVM

> **Ans: B) Native ARM machine code via its own rendering engine** — Flutter (Dart) compiles to native ARM code with its own rendering engine (Skia)
> *Trap:* Not JS-interpreted or WebView-based

110. An "implicit intent" in Android:
A) Names the exact target component by class B) Declares a general action and lets the OS resolve a suitable handling component C) Is used only for starting services D) Cannot carry any data

> **Ans: B) Declares a general action and lets the OS resolve a suitable handling component** — Implicit intent declares an action; OS resolves the handler(s)
> *Trap:* Explicit intent names the exact component/class

111. What is the file extension of a signed, installable Android application package?
A) `.exe` B) `.apk` C) `.jar` D) `.dmg`

> **Ans: B) `.apk`** — `.apk` = Android Package, the installable signed archive
> *Trap:* `.jar`/`.exe`/`.dmg` are unrelated package formats

112. Local SQLite storage on Android is most commonly accessed today through which abstraction library?
A) Realm only B) Room C) Core Data D) Retrofit

> **Ans: B) Room** — Room is Android's official SQLite abstraction layer (Jetpack)
> *Trap:* Core Data is iOS's equivalent, not Android's

Block 13 — AI (Q113–116)

113. In propositional/first-order logic used for AI knowledge representation, "Modus Ponens" allows inferring:
A) ¬P from P → Q B) Q from P and P → Q C) P from Q D) Nothing valid

> **Ans: B) Q from P and P → Q** — Modus Ponens: from P and (P→Q), infer Q
> *Trap:* Not the same as denying the antecedent

114. Inductive learning generalises a hypothesis primarily from:
A) A single labelled example B) A set of specific examples to a general rule C) An expert-provided rule base only D) Random search alone

> **Ans: B) A set of specific examples to a general rule** — Inductive learning generalises FROM specific examples TO a general rule
> *Trap:* Opposite direction from deduction

115. In PROLOG, a program is fundamentally composed of:
A) Sequential imperative statements B) Facts and rules, queried by unification and backtracking C) Only mathematical functions D) Object classes and methods

> **Ans: B) Facts and rules, queried by unification and backtracking** — PROLOG programs = facts + rules, resolved via unification/backtracking
> *Trap:* Not imperative sequencing

116. Which of these best describes "explanation-based learning" (EBL)?
A) Learning purely from statistical correlation in big data B) Using a single example plus prior domain knowledge to generalise a learned rule via deductive analysis C) Unsupervised clustering D) Random trial and error

> **Ans: B) Using a single example plus prior domain knowledge to generalise a learned rule via deductive analysis** — EBL uses one example plus background domain theory to deductively generalise a rule
> *Trap:* Not purely statistical, not unsupervised clustering

Block 14 — Machine Learning (Q117–120)

117. K-Nearest Neighbours is an example of:
A) An unsupervised clustering algorithm B) A supervised, instance-based (lazy) learning algorithm C) A reinforcement learning algorithm D) A dimensionality reduction technique

> **Ans: B) A supervised, instance-based (lazy) learning algorithm** — KNN: supervised, "lazy"/instance-based (no explicit training phase, decides at query time)
> *Trap:* Not unsupervised, not RL

118. K-means clustering minimises which objective?
A) Cross-entropy loss B) Within-cluster sum of squared distances to the centroid C) Number of clusters D) Margin between classes

> **Ans: B) Within-cluster sum of squared distances to the centroid** — K-means minimises within-cluster sum of squared distances to centroids
> *Trap:* Not a classification-margin or entropy measure

119. Bagging (Bootstrap Aggregating) primarily reduces:
A) Bias B) Variance, by averaging predictions of models trained on bootstrapped samples C) Training time only D) The need for any labelled data

> **Ans: B) Variance, by averaging predictions of models trained on bootstrapped samples** — Bagging averages many models trained on bootstrap samples → reduces variance
> *Trap:* Boosting (not bagging) targets bias via sequential correction

120. Boosting (e.g. AdaBoost) builds an ensemble by:
A) Training all models independently and averaging B) Sequentially training models, each focusing more on the previous model's errors C) Randomly discarding weak learners D) Clustering the training data first

> **Ans: B) Sequentially training models, each focusing more on the previous model's errors** — Boosting trains sequentially, weighting/focusing on prior errors
> *Trap:* Bagging (not boosting) trains independently in parallel

---

PART 2 — DESCRIPTIVE QUESTION BANK

Model-answer **skeletons only** — the bullet points an examiner is scanning for, not full prose. Expand each into full sentences in your own practice attempts; in the exam, write the expansion, not the bullet list.

PAPER-I — Descriptive Bank

Five-mark skeletons (22 — pick any 8 of 10 offered on the day)

1. State the essential properties of an algorithm.

> • Finiteness, Definiteness, Input, Output, Effectiveness — one line each.

2. Differentiate a priori and a posteriori analysis.

> • A priori = theoretical, machine-independent, before running; a posteriori = empirical, after running, machine-dependent. One-line table.

3. Define Big-O, Big-Omega and Big-Theta.

> • Formal $\exists c, n_0$ definitions for each; state $\Theta \iff O$ and Ω both hold; one growth-rate example each.

4. Explain best, worst and average case using linear search.

> • Best $\Omega(1)$ (first element); Worst $O(n)$ (last/absent); Average $\Theta(n)$, $(n+1)/2$ comparisons.

5. Differentiate merge sort and quick sort (space, stability).

> • Merge: $\Theta(n)$ aux, stable. Quick: $O(\log n)$ aux average, not stable. One-line reason each.

6. What is the Master Theorem? State its three cases.

> • Form T(n)=aT(n/b)+f(n); compare f(n) to n^(log_b a); Case 1/2/3 result each.

7. Define P, NP, NP-hard, NP-complete.

• P solved poly time; NP verified poly time; NP-hard at least as hard as NP; NP-complete = in NP and NP-hard.

8. Explain the RAM model of computation.

• Single processor, unit-cost basic ops, no memory hierarchy/concurrency; why it makes analysis machine-independent.

9. What are symbol tables? Name two implementations.

• Store name-attribute pairs (identifiers, types, scope); implementations: linear list, hash table, BST.

10. State Kirchhoff/Boolean laws — De Morgan's theorems.

• (AB)' = A'+B'; (A+B)' = A'B'; one truth-table row to justify.

11. Explain 1's and 2's complement representation of negative numbers.

• 1's: invert bits, end-around carry. 2's: invert + 1, no such carry adjustment; 2's complement is standard in modern hardware.

12. What is a K-map? What problem does it solve?

• Graphical tool to minimise SOP/POS Boolean expressions by grouping adjacent 1s (or 0s) in powers of two.

13. Differentiate combinational and sequential circuits.

• Combinational: output depends only on current inputs, no memory. Sequential: output depends on inputs + present state (has memory/clock).

14. What is pipelining? State one hazard type.

• Overlap fetch/decode/execute of successive instructions; hazards: structural, data, control — describe one.

15. Differentiate direct and associative cache mapping.

• Direct: fixed one-to-one block-to-line mapping, fast but conflict-prone. Associative: any block anywhere, flexible but costlier hardware.

16. What is DMA? Why is it used?

• Direct Memory Access transfers data to/from memory without CPU handling every word — frees CPU for other work during I/O.

17. Differentiate a compiler and an interpreter.

• Compiler translates whole program to machine code before execution; interpreter executes line-by-line at runtime, no standalone executable produced.

18. What is a macro? How is it different from a function?

• Macro expanded inline at compile/assembly time (no call overhead, code duplicated); function called at runtime (call/return overhead, code shared).

19. State the phases of a compiler in order.

• Lexical → Syntax → Semantic → Intermediate code → Optimisation → Code generation.

20. What factors constrain mobile application development?

• Battery, network, screen size/density, input method (touch), fragmentation, interruptions — name and justify five.

21. Differentiate native and hybrid mobile applications.

• Native: platform language, best performance, per-OS codebase. Hybrid: one shared codebase (React Native/Flutter), near-native performance.

22. What is text-to-speech? Name its stages.

• Text analysis/normalisation → linguistic analysis (prosody) → waveform synthesis (concatenative/parametric/neural).

Ten-mark skeletons (14 — pick any 3 of 5 offered on the day)

1. Solve T(n) = 2T(n/2) + n by (a) recursion tree (b) Master theorem.

• Tree: n work per level × log n levels = Θ(n log n). Master: a=2,b=2,f(n)=n,e=1, Case 2 → Θ(n log n). Show both derivations fully.

2. Explain the working of Quick Sort. State its average and worst-case complexity. *(2025 exact question)*

• Partition step (Lomuto/Hoare) with worked trace on a small array; average Θ(n log n) balanced splits; worst Θ(n^2) on sorted input with poor pivot; fixes: randomised/median-of-three pivot.

3. Solve 0/1 Knapsack by dynamic programming for a given instance.

• Build the DP table dp[i][w]; recurrence dp[i][w]=max(dp[i-1][w], v_i+dp[i-1][w-w_i]); fill for the given items/capacity; read optimal value from the final cell; trace back selected items.

4. Explain Prim's or Kruskal's algorithm for Minimum Spanning Tree with a worked example.

• State greedy criterion (Kruskal: sort edges ascending, add if no cycle via union-find; Prim: grow tree, always add cheapest edge crossing the frontier); trace on a 5–6 node graph; total MST weight.

5. Explain P vs NP vs NP-complete vs NP-hard with a diagram and one example each.

• Venn-diagram description: P ⊆ NP; NP-complete = NP ∩ NP-hard; example NP-complete: SAT/TSP-decision; state Cook's theorem names SAT as the first proven NP-complete problem.

6. Design and simplify a combinational circuit for a given Boolean function using K-map.

• Draw truth table → K-map → group adjacent 1s in largest power-of-2 blocks → read off minimal SOP → draw the gate-level circuit.

7. Explain the phases of a compiler with a diagram, using a sample statement.

• Trace `a = b + c * d` through lexical (tokens) → syntax (parse tree) → semantic (type check) → intermediate code (3-address code) → optimisation → target code.

8. Explain the Android activity lifecycle with a diagram.

> • Draw the state diagram (onCreate→onStart→onResume→onPause→onStop→onDestroy, plus onRestart loop); explain why it exists (interruptions, OS memory reclamation).

9. What are the four Android components? Explain each with an example.

> • Activity (UI screen), Service (background task, e.g. music player), Broadcast Receiver (system event listener), Content Provider (shared structured data access).

10. Differentiate explicit and implicit intents; give one example each.

> • Explicit: names target class, e.g. launching your own SettingsActivity. Implicit: declares action (ACTION_SEND), OS resolves candidate apps.

11. Discuss the frameworks and tools used in mobile app development. *(2025 exact question)*

> • IDEs (Android Studio, Xcode); cross-platform (React Native, Flutter, Ionic); storage (SQLite/Room/Core Data); UI guidelines (Material/HIG); testing (Espresso/XCTest/Appium); crash reporting (Crashlytics/Sentry).
> **12. Explain synchronization and replication of mobile data.**
> • Define both terms; conflict scenarios (offline edit + server edit); resolution strategies (last-write-wins, manual merge); offline-first design principle.

13. Explain the packaging and deployment process for an Android app.

> • Build → package into APK/AAB → sign with private key → version (`versionCode`/`versionName`) → upload to Play Console → review → staged rollout.

14. Explain how a floating-point number is represented (IEEE-754 single precision).

> • Sign bit, 8-bit exponent (biased by 127), 23-bit mantissa; worked example converting a small decimal to its bit pattern.

Fifteen-mark skeletons (8 — pick any 2 of 4 offered on the day)

1. Describe and compare divide-and-conquer and greedy paradigms, with a concrete example of each. *(2025 exact question)*

> • D&C: divide/conquer/combine steps; example merge sort, recurrence + Master theorem solve. Greedy: candidate/selection/feasibility/objective functions; example fractional knapsack, worked table. Comparison table: where the work sits, correctness guarantee (exchange argument vs recursive correctness), when each applies (independent subproblems vs greedy-choice + optimal substructure).
> **2. Explain dynamic programming with two fully worked examples (0/1 Knapsack and LCS).**
> • Define overlapping subproblems + optimal substructure; DP table for knapsack (from Paper-I bank above); DP table for LCS with traceback reading off the actual subsequence; contrast with D&C (no overlap) and greedy (no correctness guarantee here).

3. Design a MOD-10 (BCD) synchronous counter. Draw the state diagram, state table and logic circuit.

• 4 flip-flops needed (⌈log₂10⌉=4); state diagram 0000→...→1001→0000 (10 states, skip 1010–1111); state table with next-state and flip-flop input equations (derive JK/D inputs via K-map); draw the resulting circuit.

4. Explain the complete compiler pipeline with a running example, including code optimisation.

• Full pipeline diagram; trace one small program through lexical/syntax/semantic/intermediate/optimisation/codegen; name at least two optimisation techniques (constant folding, dead code elimination, loop-invariant code motion) with a before/after snippet.

5. Discuss the challenges in designing the right UI for mobile applications. *(2025 exact question)*

• Screen fragmentation → responsive/dp-based layout; touch imprecision → 48dp targets; platform convention → Material/HIG; interruption-prone context → short tasks, high contrast, thumb-zone reachability; close with the one-line thesis (obvious/reachable/reversible next action).

6. Explain CPU addressing modes with an example instruction for each, and their trade-offs.

• Immediate, direct, indirect, register, register-indirect, indexed, base-register — one example instruction and one trade-off (speed vs flexibility) per mode.

7. Explain Strassen's matrix multiplication algorithm; derive and compare its complexity with the conventional method.

• Naive Θ(n³); block-decompose into 8 vs Strassen's 7 products; write all 7 P-terms and the 4 C-block reconstructions; Master theorem derivation to Θ(n^2.807); honest limitations (constant factor, additions overhead, crossover point).

8. Discuss NP-completeness, Cook's theorem, and reduction technique, with SAT as example.

• Define P/NP/NP-hard/NP-complete; state Cook-Levin theorem (SAT is NP-complete); explain polynomial-time reduction as the proof technique for showing a new problem is NP-complete (reduce a known NP-complete problem to it); significance of P=NP question.

PAPER-II — Descriptive Bank

Five-mark skeletons (22 — pick any 8 of 10 offered on the day)

1. Differentiate multiprogramming and multitasking.

• Multiprogramming: several jobs resident in memory, CPU switches on I/O wait, no user-interaction focus. Multitasking: rapid switching giving illusion of simultaneous execution, typically interactive/time-shared.

2. State the necessary conditions for deadlock.

• Mutual exclusion, Hold-and-wait, No preemption, Circular wait — all four must hold simultaneously.

3. Differentiate paging and segmentation.

> • Paging: fixed-size physical blocks, no external fragmentation, may have internal fragmentation. Segmentation: variable-size logical units, may have external fragmentation, matches program's logical structure.

4. What is Belady's anomaly?

> • Under FIFO, page faults can INCREASE when the number of frames increases — counter to intuition.

5. Differentiate verification and validation.

> • Verification: are we building it right (per spec)? Validation: are we building the right thing (per user needs)?

6. What is cyclomatic complexity? State its formula.

> • V(G) = E − N + 2P (edges, nodes, connected components); measures independent linear paths through control flow.

7. Differentiate black-box and white-box testing.

> • Black-box: behaviour only, no code knowledge, input/output based. White-box: internal code structure known, tests specific paths/branches.

8. What is normalization? Define 1NF.

> • Process of organising data to reduce redundancy/anomalies; 1NF: atomic values only, no repeating groups.

9. Differentiate DELETE, TRUNCATE and DROP in SQL.

> • DELETE: removes rows, can be conditional, logged, rollback-able. TRUNCATE: removes all rows fast, minimal logging. DROP: removes the entire table structure.

10. What is a foreign key? What integrity does it enforce?

> • Column(s) referencing a primary/unique key of another table; enforces referential integrity (no orphaned references).

11. Differentiate LAN, MAN and WAN.

> • LAN: small area (building/campus). MAN: city-scale. WAN: country/global scale, often uses leased/public infrastructure.

12. State the layers of the OSI model in order.

> • Physical, Data Link, Network, Transport, Session, Presentation, Application (bottom to top).

13. Differentiate TCP and UDP.

> • TCP: connection-oriented, reliable, ordered, higher overhead. UDP: connectionless, unreliable, low overhead, used for streaming/DNS.

14. What is DNS? Why is it needed?

> • Domain Name System translates human-readable domain names to IP addresses; needed because humans use names, routers use numeric addresses.

15. Explain the basic HTML document structure.

> • `<!DOCTYPE html>`, `<html>` root, `<head>` (metadata/title), `<body>` (visible content) — name each with one purpose.

16. Differentiate GET and POST methods.

> • GET: data in URL query string, visible, size-limited, idempotent. POST: data in request body, hidden, larger limits, used for state-changing submissions.

17. What are PHP sessions? Where is session data stored?

> • Mechanism to persist data across requests for one user; data stored server-side, only the session ID travels via a client cookie.

18. Differentiate client-side and server-side scripting.

> • Client-side (JavaScript): runs in browser, no DB access. Server-side (PHP): runs on server, generates HTML, has DB access, source never sent to client.

19. Explain the CSS box model.

> • Content → Padding → Border → Margin, innermost to outermost; briefly describe each layer's role.

20. State the phases of website development.

> • Planning, Design, Implementation (front-end + back-end), Testing, Deployment, Maintenance — the syllabus stresses Implementation/Testing/Maintenance.

21. What is inductive learning? Give an example.

> • Generalising a rule from specific labelled examples; example: learning "all observed swans are white" from a set of swan sightings.

22. Differentiate supervised and unsupervised learning.

> • Supervised: labelled data, learns input→output mapping (e.g. KNN, regression). Unsupervised: unlabelled data, finds structure (e.g. K-means clustering).

Ten-mark skeletons (14 — pick any 3 of 5 offered on the day)

1. Explain any two CPU scheduling algorithms with a Gantt chart and average waiting time calculation.

> • Pick FCFS and SJF (or Round Robin); given arrival/burst times, draw Gantt chart, compute completion/turnaround/waiting times, average them; note starvation/context-switch trade-offs.

2. Explain the necessary conditions for deadlock and one deadlock-prevention strategy for each.

> • List all four conditions; for each, name a concrete OS technique to violate it (e.g. resource ordering breaks circular wait; requesting all resources upfront breaks hold-and-wait).

3. Explain paging with address translation, using a numeric example.

> • Logical address split into page number + offset; page table maps page→frame; worked example translating a given logical address to a physical address.

4. Compare FIFO, LRU and Optimal page replacement algorithms on a given reference string.

> • Trace all three algorithms on the same reference string with a given number of frames; count page faults for each; conclude Optimal ≤ LRU ≤ FIFO (in general).

5. Explain the Waterfall and Spiral SDLC models, and compare them.

> • Waterfall: sequential phases, diagram; Spiral: repeated risk-driven cycles, diagram; comparison table (flexibility, risk handling, when each suits a project).

6. Explain the E-R model with a diagram, converting it to a relational schema.

> • Entities, attributes, relationships (1:1, 1:M, M:N) in an E-R diagram for a sample scenario (e.g. Student-Course); convert to tables with PK/FK, noting how M:N needs a junction table.

7. Write SQL to create a table, insert records, and answer 2–3 queries (JOIN, GROUP BY, subquery).

> • `CREATE TABLE` with constraints; `INSERT` sample rows; queries using `JOIN`, `GROUP BY ... HAVING`, and a subquery; show expected output rows for each.

8. Explain the OSI reference model layer by layer with the function/protocol example of each.

> • Seven layers, one function + one protocol/example per layer (e.g. Network: IP; Transport: TCP/UDP; Application: HTTP/FTP).

9. Differentiate and explain the working of a router, switch and hub.

> • Hub: physical-layer broadcaster, no intelligence. Switch: data-link layer, MAC-address-based forwarding, builds a MAC table. Router: network-layer, IP-based routing between different networks.

10. Explain the difference between GET and POST methods in HTML forms. *(2025 exact question)*

> • Full comparison table (location of data, superglobal, visibility, size limit, idempotency, suitable use cases); short PHP code snippet for each showing `$_GET`/`$_POST` retrieval.
> **11. What are the fundamental syntax rules and use of variables in PHP scripting?** *(2025 exact question)*
> • `$` prefix, no type declaration, dynamic typing, case sensitivity, `.` for concatenation, embedding via `<?php ?>`, `.php` extension requirement; short worked code example with traced output.
> **12. Write a PHP script to connect to MySQL and display records from a table; explain each step.**
> • `mysqli_connect()` (host, user, pass, db) → check connection → `mysqli_query()` → `mysqli_fetch_assoc()` loop → `mysqli_close()`; trace expected output for 2–3 sample rows.

13. Explain inductive learning and explanation-based learning, contrasting the two.

> • Inductive: generalise from many examples statistically/empirically. EBL: generalise from one example using existing domain theory, deductively; state which needs more prior knowledge vs more data.

14. Explain K-means clustering with a worked numeric example (2–3 iterations).

> • Choose k initial centroids; assign points to nearest centroid; recompute centroids; repeat for 2 iterations on a small 2D point set; show convergence / stopping condition.

Fifteen-mark skeletons (8 — pick any 2 of 4 offered on the day)

1. Explain process scheduling, and derive average waiting/turnaround time for FCFS, SJF and Round Robin on the same data set.

> • One data set (arrival + burst times) run through all three algorithms; three Gantt charts; comparison table of averages; conclude with trade-off discussion (fairness vs throughput vs starvation).

2. Explain memory management techniques: paging, segmentation, and virtual memory with demand paging.

> • Paging (fixed frames, page table, internal fragmentation); Segmentation (logical variable-size units, external fragmentation); Virtual memory/demand paging (pages loaded on demand, page fault handling, thrashing risk); one diagram per technique.

3. Discuss the SDLC in detail: waterfall, spiral, and agile/prototype models, with when to use each.

> • Diagram + description for each of the three-plus models; a decision guide (well-understood requirements → waterfall; high risk/uncertainty → spiral; changing requirements → agile).

4. Explain normalization up to BCNF with a worked example table taken through each normal form.

> • Start with an unnormalised table; show partial dependency removed (1NF→2NF); transitive dependency removed (2NF→3NF); BCNF condition (every determinant is a candidate key) checked and fixed if violated; show the final decomposed schema.

5. Explain the OSI and TCP/IP models, mapping layers between them, with one protocol example per layer.

> • Seven OSI layers vs four/five TCP/IP layers, mapping table; one protocol example named per layer; explain why TCP/IP became the practical standard despite OSI being the formal reference model.

6. Describe the phases of website development, from planning to maintenance. *(2025 exact question)*

> • Planning/requirements → Design (responsive, interactive states) → Implementation (front-end HTML/CSS/JS, back-end PHP+MySQL/Node/Django) → Testing (functional, cross-browser, security) → Deployment → Maintenance (continuous, unlike shrink-wrapped software); state the "never finished" thesis explicitly.

7. Explain supervised, unsupervised and ensemble learning methods (KNN, K-means, Bagging/Boosting) with one example each.

> • KNN: instance-based classification example. K-means: clustering example with centroid iteration. Bagging vs Boosting: variance vs bias reduction, one real algorithm named each (Random Forest for bagging, AdaBoost for boosting).

8. Explain database transactions, the ACID properties, and concurrency control techniques.

> • Define Atomicity, Consistency, Isolation, Durability with a one-line example failure each property prevents; concurrency control: locking (shared/exclusive), timestamp ordering; brief mention of deadlock in the DB context (parallels OS deadlock).

Cross-check against 2025 papers — calibration note

Every question marked *(2025 exact question)* above is a direct match — in topic and mark value — to a question that actually appeared in the CTSE 2025 CS-Degree Paper-I or Paper-II, per `_raw/existing_txt/CTSE 2025 - Paper I - Questions and Answers.txt` and `CTSE 2025 - Paper II - Questions and Answers.txt`. If you can produce all of those answers from a blank page, at the stated mark's depth, you have matched the 2025 candidate's preparation on the exact points that were tested last year — treat everything else in this bank as the reasonable extension of that same syllabus coverage into adjacent, equally-likely topics.